

A decorative graphic on the left side of the slide, consisting of a vertical line of small circles connected by horizontal and diagonal lines, resembling a circuit board or a stylized tree structure.

DISEÑO DE SISTEMAS INFORMÁTICOS II

REUTILIZACIÓN DE SOFTWARE COMPONENTES REUTILIZABLES

Escuela Superior Técnica

REUTILIZACIÓN DE SOFTWARE

- "Reutilización de software es el proceso de crear sistemas de software a partir de software existente, en lugar de desarrollarlo desde el comienzo" (Sametinger, 1997)
- La reutilización es un enfoque de desarrollo [de software] que trata de maximizar el uso recurrente de componentes de software existentes" (Sommerville, 1995).
- "La reutilización de software es el proceso de implementar o actualizar sistemas de software usando activos de software existentes" (Sodhi & Sodhi, 1999)

REUTILIZACIÓN DE SOFTWARE

- Utilizada como técnica: Copiar y pegar.
- No es una reutilización profesional.
- Es necesario conocer el código anterior.
- Se duplica el mantenimiento posterior.
- No genera documentación a futuro.

REUTILIZACIÓN DE SOFTWARE

- La reutilización de software es un proceso de la Ingeniería de Software que involucra el uso recurrente de activos de software en:
 - la especificación,
 - el análisis,
 - el diseño,
 - la implementación y
 - las pruebas de una aplicación o sistema de software

REUTILIZACIÓN DE SOFTWARE

- ¿Qué es un activo de software reutilizable (ASR)?
 - Son productos de software diseñados expresamente para ser reutilizados en el desarrollo de muchas aplicaciones
 - Un activo de software puede ser:
 - Un componente de software
 - Una especificación de requerimientos
 - Un modelo o especificación de diseño
 - Un algoritmo
 - Un patrón de diseño
 - Una arquitectura de dominio
 - Un esquema de base de datos
 - Un lote de casos de prueba
 - La documentación de un sistema
 - Un plan

COMPONENTES DE SOFTWARE REUTILIZABLES

- Un componente de software reutilizable (CSR) es un artefacto de software auto contenido y claramente identificable que:
 - ejecuta funciones específicas,
 - tiene una interfaz clara a través de la cual se integra a otros sistemas,
 - tiene una documentación apropiada y
 - tiene un status de reuso definido
- Un componente de software reutilizable puede ser:
 - un módulo,
 - una clase,
 - un procedimiento o función,
 - un subsistema,
 - una aplicación

COMPONENTES DE SOFTWARE REUTILIZABLES

- Un componente es una pieza de software que describe y/o libera un conjunto de servicios que son usados sólo a través de interfaces definidas
- Características esenciales de un CSR:
 - Identificable
 - Autocontenido
 - Rastreable a través de su ciclo de desarrollo
 - Reemplazable por otro componente
 - Accesible solamente a través de su interfaz
 - Inmutabilidad de sus servicios
 - Documentación de sus servicios
 - Mantenido sistemáticamente

CARACTERÍSTICAS DE UN CSR

- Es identificable
 - Debe tener una identificación clara y consistente que facilite su catalogación y acceso.
- Es autocontenido
 - Un componente no debe requerir la reutilización de otros componentes para cumplir su función.
 - Si el componente requiere de otros, el conjunto de componentes o funciones, visto como un todo, debe ser considerado como un componente de más alto nivel.
 - P.ej., los paquetes en Java permiten agrupar varias clases en un componente al cual se le asocia una funcionalidad bien definida.
- Es rastreable
 - Debe retener su identidad y ser rastreable durante el ciclo de desarrollo.

CARACTERÍSTICAS DE UN CSR

- Es reemplazable
 - Puede ser reemplazado por una nueva versión o por otro componente que proporcione los mismos servicios
- Es accesible sólo a través de su interfaz
 - Es accedido a través de una interfaz claramente definida
 - La interfaz debe ser independiente de la implementación física
 - debe ocultar los detalles de su diseño interno
- Sus servicios son inmutables
 - Los servicios que ofrece un componente a través de su interfaz no deben variar
 - La implementación física de estos servicios puede ser modificada, pero no debe afectar la interfaz

CARACTERÍSTICAS DE UN CSR

- Está documentado
 - Debe tener una documentación adecuada que facilite:
 - la recuperación del componente desde el repositorio,
 - la evaluación del componente,
 - su adaptación al nuevo ambiente y
 - su integración con otros componentes del sistema en que se reutiliza.

CARACTERÍSTICAS DE UN CSR

- Es mantenido
 - Debe ser mantenido para facilitar un reuso sistemático:
 - Su estado de reuso debe contener información acerca de:
 - quien es el propietario del componente,
 - quien lo mantiene,
 - a quien acudir cuando surjan problemas y
 - cual es el estado de la calidad del componente

CARACTERÍSTICAS DESEABLES

- Es genérico
- Su implementación física está oculta
- Es independiente de otros componentes
- Está encapsulado
- Es independiente del lenguaje o a las herramientas usadas en su desarrollo
- Es independiente de la plataforma de ejecución
- Puede ser reusado dinámicamente
- Está certificado

INTERFAZ DE UN COMPONENTE

- Una interfaz determina la manera en que un componente se reutiliza y como puede conectarse a otros componentes
- Una interfaz define una operación o un conjunto de operaciones llamadas servicios o responsabilidades
- Las interfaces son un aspecto fundamental en la composición de componentes
 - Por lo general un componente produce (exporta) un resultado que es consumido (importado) por otro componente

INTERFAZ DE UN COMPONENTE

- La naturaleza de la interfaz varia dependiendo del lenguaje empleado para implementar el componente
- En lenguajes OO:
 - las clases se diseñan de tal manera que la interfaz sea visible y separada de la implementación de las operaciones (métodos)
- En lenguajes procedimentales:
 - la interfaz se define a través de:
 - La declaración de la función o procedimiento o
 - El uso de variables globales

COMPOSICIÓN DE SOFTWARE

- La composición de software es "el proceso de construir aplicaciones mediante la interconexión de componentes de software a través de sus interfaces [de composición]".
- Composición Orientada a Objetos:
 - Basada en la reutilización de clases o tipos escritos en un lenguaje OO
 - Emplean varios mecanismos de reutilización: herencia, delegación, polimorfismo y encadenamiento dinámico
 - Ejemplos de componentes OO:
 - La colección de clases reutilizables del lenguaje Eiffel
 - La librería de clases GNU para C++
 - La librería del sistema en el lenguaje Java

COMPOSICIÓN DE SOFTWARE

- Arquitecturas de Objetos Distribuidos
 - Fueron desarrolladas para facilitar la computación distribuidas en sistemas heterogéneos
 - Constituyen la base para la interoperabilidad de componentes distribuidos en plataformas heterogéneas
 - Ejemplos:
 - CORBA del consorcio OMG
 - Component Object Model (COM) y DCOM (Distributed COM) de Microsoft
 - System Object Model (SOM) de IBM
 - Enterprise JavaBeans de SUN

PROCESO DE REUTILIZACIÓN DE SOFTWARE

- Los procesos de desarrollo basados en la reutilización de software se clasifican en:
 - Desarrollo de componentes de software reutilizables
 - Adaptación o desarrollo de componentes con el propósito expreso de ser reutilizados en futuras aplicaciones
 - Desarrollo de software con reutilización de componentes
 - El desarrollo de una nueva aplicación involucra el reuso de un conjunto de componentes existentes

DESARROLLO DE UN COMPONENTE REUTILIZABLE

- El proceso de adaptación de un componente de software:
 - Generalización del nombre
 - El nombre del componente debe referirse a entidades genéricas del dominio.
 - Generalización de operaciones
 - La funcionalidad del componente puede ser extendida agregando nuevas operaciones.
 - Las operaciones específicas deben ser removidas.

DESARROLLO DE UN COMPONENTE REUTILIZABLE

- Generalización de excepciones
 - Las excepciones específicas de una aplicación deben ser removidas.
 - Los mecanismos de manejo de excepciones se agregan al componente para incrementar su robustez.
- Certificación del componente
 - El componente es certificado como un componente reutilizable.
 - Involucra asegurar la calidad y confiabilidad del componente.

DESARROLLO DE SOFTWARE CON REUTILIZACIÓN DE COMPONENTES

- Desarrollo de software con reutilización de componentes
 - Es un enfoque de desarrollo de software que
 - maximiza la reutilización de componentes software existentes y/o
 - reduce el número de componentes que requieren ser desarrollados desde el comienzo
 - Condiciones mínimas para la reutilización
 - Existencia de repositorios o bases de componentes reutilizables
 - Los componentes son confiables y actúan de acuerdo a sus especificaciones

UTILIDAD DE LA REUTILIZACIÓN DE SOFTWARE

- Varios estudios han demostrado la efectividad de la reutilización del software:
 - 40-60% del código fuente es reutilizable de una aplicación a otra.
 - Aproximadamente el 60% del diseño y del código de aplicaciones administrativas es reutilizable.
 - Aproximadamente el 75% de las funciones son comunes a más de un programa.
 - Sólo el 15% del código encontrado en muchos sistemas es único y novedoso a una aplicación específica.
 - El rango general de reuso potencial está entre el 15% y el 85%.

BENEFICIOS

- “La reutilización es la única aproximación realista para llegar a los índices de productividad y calidad que la industria del software necesita” (Mili et al. 95).
- Mejora de la productividad:
 - Disminución tiempo de desarrollo:
⇒ *mejor adaptación requisitos cambiantes*
¡Los requisitos no son estables!
 - Disminución de costos
- Mejora de la calidad del software:
 - Mayor fiabilidad
 - Mayor eficiencia (aunque al principio pueda parecer que no)

OBSTÁCULOS PARA REUTILIZAR

- En muchas organizaciones no existe plan de reutilización (no se considera prioritario)
- Escasa formación
- Resistencia del personal
- Pobre soporte metodológico
 - uso de métodos que no promueven la reutilización (estructurados)
 - Necesarios métodos para:
 - desarrollo para reutilización
 - desarrollo con reutilización
- *¿Quién financia los gastos iniciales de la reutilización?*

BIBLIOGRAFÍA

- www.cbdiforum.com
- Consorcio de empresas de desarrollo de software orientadas hacia la reutilización.
- www.corba.org
- Modelo de arquitectura distribuida para interacción de componentes.